

Taller 2

Reflexión sobre el trabajo realizado



ISIS-1611

Autores:

Martin De Angulo – 202421628

Yara Gutierrez – 202511181

Mariana Rodríguez – 202421258

Daniel Vergara – 202320392

Universidad de los Andes

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas y Computación

Algoritmos CSP	3
Complejidad Temporal:.....	3
Complejidad Espacial:.....	3
Reducción de búsqueda:.....	4
Comparativa cuantitativa.	4
Búsqueda Adversaria:	6
Complejidad Temporal y Espacial de Minimax:	6
¿Por qué Minimax puede ser pesimista?:	6
Función de evaluación:	7
Comparativa Tasas de victoria:	7
Poda Alfa-Beta:	8
Reflexión final:	8

Algoritmos CSP

Complejidad Temporal:

- a. **Backtracking_search**: El backtracking básico tiene una complejidad $O(d^n)$ en el peor caso, donde: n es el número de variables (entregas) y d es el tamaño máximo del dominio (número de drones). Esto ocurre cuando el algoritmo debe explorar todas las combinaciones posibles antes de encontrar una solución o determinar que no existe. Una búsqueda exhaustiva sin backtracking tendría exactamente la misma complejidad $O(d^n)$ pero sin ninguna poda, ya que generaría y verificaría todas las combinaciones completas.
- b. **Backtracking_fc**: Forward Checking tiene la misma complejidad teórica $O(d^n)$ en el peor caso, pero en la práctica es significativamente más eficiente porque elimina valores inconsistentes de los dominios de los vecinos después de cada asignación, reduciendo el árbol explorado antes de llegar a los nodos hoja.
- c. **Backtracking_ac3**: AC-3 tiene una complejidad temporal $O(n^2 \cdot d^3 \cdot d^n)$, en el peor de los casos donde: n es el número de variables (entregas) y d es el tamaño del dominio (drones). Ya que cada asignación se vuelve a ejecutar con AC-3 sobre los vecinos de la variable asignada. Sin embargo, en la práctica este peor caso rara vez se alcanza porque AC-3 reduce los dominios globalmente antes y durante la búsqueda.
- d. **Backtracking_mrv_lcv** (bono): En el peor caso el algoritmo sigue siendo exponencial, como cualquier backtracking para CSP. Si hay n variables y cada una tiene a lo sumo d valores en su dominio, el peor caso es aproximadamente $O(d^n)$ porque puede explorar casi todas las combinaciones posibles. Las heurísticas **MRV** y **LCV** no cambian el peor caso teórico, pero en la práctica reducen mucho el número de ramas exploradas. Además, el forward checking agrega un costo extra de revisar vecinos en cada asignación.

Complejidad Espacial:

- a. **Backtracking_search**: El backtracking básico tiene complejidad en espacio $O(n)$ donde n es el número de variables. Esto se debe a que el algoritmo solo almacena la asignación parcial actual en el stack de recursión, cuya profundidad máxima es n .

- b. Backtracking_fc: Forward Checking tiene complejidad en espacio $O(n \cdot d)$ porque además de la asignación parcial almacena, para cada nivel de recursión, el diccionario de valores eliminados de los dominios vecinos, que en el peor caso puede contener d valores para cada una de las n variables.

Para los algoritmos anteriores, el número de restricciones c no afecta directamente la complejidad en espacio, pero sí influye en el tiempo de ejecución, ya que la función `is_consistent()` debe evaluar todas las restricciones aplicables en cada verificación.

- c. Backtracking_ac3: AC-3 tiene una complejidad espacial $O(n \cdot d \cdot n^2)$ corresponde a `saved_domains` que guarda una copia completa de todos los dominios en cada nivel de recursión para poder restaurarlos en caso de backtrack, $O(n)$ para la pila de recursión, y $O(n^2)$ para la cola de arcos de AC-3.
- d. Backtracking_mrv_lcv (bono): El espacio está dominado por la profundidad de la recursión, que en el peor caso es n (una llamada por variable). También se almacenan los dominios modificados durante el forward checking para poder restaurarlos al hacer backtrack. Por esto, el uso total de memoria es aproximadamente $O(n \cdot d)$ considerando dominios y estructuras auxiliares. En la práctica suele ser bajo porque solo se guarda información de la rama actual del árbol de búsqueda.

Reducción de búsqueda:

Una búsqueda exhaustiva simple generaría todas las d^n asignaciones completas y verificaría cada una de ellas.

El backtracking básico mejora esto podando ramas del árbol en cuanto detecta una inconsistencia parcial, sin necesidad de completar la asignación.

Forward Checking mejora aún más este proceso porque detecta inconsistencias antes de asignar la siguiente variable, eliminando valores inválidos de los dominios de los vecinos y evitando explorar ramas que inevitablemente fallarán.

El Backtracking con AC-3 lleva la reducción del espacio de búsqueda un paso más allá que Forward Checking. Mientras que FC solo elimina valores inconsistentes de los vecinos directos de la variable recién asignada, AC-3 propaga en cascada por toda la red: si eliminar un valor de X_i reduce el dominio de X_j , vuelve a revisar todos los vecinos de X_j , y así sucesivamente. Además, AC-3 se ejecuta antes de iniciar la búsqueda, reduciendo dominios globalmente desde el principio, lo que puede eliminar valores inválidos antes de explorar un solo nodo.

Comparativa cuantitativa.

- Layout 1: `twin_bases`

Algoritmo	Tiempo (s)	#asignaciones	Solución encontrada (SI/NO)
Search	0.0000s	5	SI
Forward_Checking	0.0000s	3	SI
AC-3	0.0000s	2	SI
Mrv_lcv	0.0000s	2	SI

- Layout 2: tight_windows

Algoritmo	Tiempo (s)	#asignaciones	Solución encontrada (SI/NO)
Search	0.0000s	4	SI
Forward_Checking	0.0001s	3	SI
AC-3	-	-	NO
Mrv_lcv	0.0000s	3	SI

- Layout 3: tight_overload

Algoritmo	Tiempo (s)	#asignaciones	Solución encontrada (SI/NO)
Search	0.0000s	6	SI
Forward_Checking	0.0000s	4	SI
AC-3	0.0000s	4	SI
Mrv_lcv	0.0000s	4	SI

- Layout 4: big_fleet

Algoritmo	Tiempo (s)	#asignaciones	Solución encontrada (SI/NO)
Search	0.0000s	21	SI
Forward_Checking	0.0000s	7	SI
AC-3	0.0000s	6	SI
Mrv_lcv	0.0000s	7	SI

- Layout 5: extreme*

Algoritmo	Tiempo (s)	#asignaciones	Solución encontrada (SI/NO)
Search	0.0010s	46	SI
Forward_Checking	0.0005s	21	SI
AC-3	0.0020s	17	SI
Mrv_lcv	0.0000s	9	SI

- Layout 6: custom_nightmare*

Algoritmo	Tiempo (s)	#asignaciones	Solución encontrada (SI/NO)
Search	1.2235s	209824	NO
Forward_Checking	3.1246s	34331	NO
AC-3	10.8213s	132255	NO
Mrv_lcv	0.9326s	3434	NO

* Se le pidió a ChatGPT 5.4 crear dos nuevos layouts que fueran mucho más complejos de solucionar para los algoritmos. En extreme.lay todos los algoritmos encontraron una solución. Por el contrario, en custom_nightmare ninguno pudo encontrar solución (se le pidió explícitamente a ChatGPT hacer un layout casi insatisfacible).

Búsqueda Adversaria:

Complejidad Temporal y Espacial de Minimax:

La complejidad del algoritmo Minimax depende principalmente de dos factores:

- **b**: factor de ramificación (número promedio de acciones posibles por agente)
- **d**: profundidad de búsqueda

En un juego con múltiples agentes, cada nivel del árbol corresponde al turno de un agente (dron o cazador). Si existen k cazadores, cada ciclo completo del juego incluye:

- una capa MAX (dron)
- Seguida por k capas MIN (cazadores)

Por esta razón, la profundidad efectiva del árbol es: $d(1 + k)$ y la complejidad en tiempo de Minimax en el peor caso es: $O(b^{d(1+k)})$. Esto ocurre porque el algoritmo explora todas las combinaciones posibles de acciones hasta alcanzar la profundidad límite.

Ahora bien, la complejidad en espacio es menor, ya que la implementación recursiva solo necesita almacenar el camino actual del árbol. Por lo tanto, el espacio requerido es proporcional a la profundidad del árbol: $O(b(d(1 + k)))$.

¿Por qué Minimax puede ser pesimista?:

Minimax asume que los adversarios juegan de manera perfectamente racional, es decir, que siempre elegirán la acción que minimiza la utilidad del agente MAX (el dron).

Sin embargo, en muchos escenarios reales los cazadores pueden comportarse de manera subóptima o incluso aleatoria. En estos casos Minimax puede resultar pesimista, ya que evalúa cada estado considerando siempre el peor comportamiento posible del adversario.

Esto puede llevar al dron a adoptar estrategias demasiado conservadoras, evitando rutas potencialmente buenas simplemente porque existe un escenario adverso extremo, aunque ese escenario sea poco probable en la práctica.

Función de evaluación:

- Describa y justifique su función de evaluación: ¿qué características del estado usa, qué pesos asignó y por qué?

La función de evaluación combina cuatro características del estado del juego, cada una con un peso distinto justificado por su importancia relativa:

1. Puntaje actual ($\text{score} * 5$) Se incluye el puntaje del juego como base, multiplicado por 5 para que las entregas completadas tengan un impacto positivo acumulativo en la evaluación. Esto asegura que el agente valore el progreso ya realizado.
2. Proximidad a la entrega más cercana ($50 / (\text{nearest_delivery} + 1)$) Se recompensa al dron por estar cerca de una entrega pendiente usando BFS real sobre el mapa, respetando paredes y terreno. El peso de 50 es moderado para que el dron avance hacia las entregas, pero sin ignorar completamente a los cazadores. El +1 evita división por cero cuando el dron ya está en la entrega.
3. Penalización por entregas pendientes ($\text{len}(\text{deliveries}) * 40$) Cada entrega pendiente resta 40 puntos, lo que presiona al agente a completar entregas activamente. El peso de 40 es ligeramente menor que el de proximidad (50) para que el dron prefiera acercarse a una entrega antes que simplemente existir lejos de los cazadores.
4. Penalización por cazadores cercanos ($200 / \text{hunter_dist}$) Es la característica con mayor peso (200) porque la supervivencia es prioritaria sobre las entregas. Se usa BFS restringido a terreno normal ($\text{hunter_restricted}=\text{True}$) para modelar correctamente que los cazadores no pueden atravesar montañas, niebla ni tormentas. Si un cazador está a distancia ≤ 1 , retorna -1000 directamente simulando una derrota inminente.
5. Urgencia de entrega (+30) Se otorga un bonus de 30 por cada entrega que el dron puede alcanzar antes que cualquier cazador. Esto evita que el dron oscile por miedo excesivo a los cazadores cuando en realidad tiene tiempo de completar una entrega de forma segura. El peso de 30 es deliberadamente menor que la penalización por cazadores para no comprometer la supervivencia.

Comparativa Tasas de victoria:

- Layout 1: Bottleneck

Algoritmo	Wins	Losses	Avg Score
MiniMax	3/3	0/3	981.7
ExpectimaxAgent	3/3	0/3	985.0

- Layout 2: Drone_escape

Algoritmo	Wins	Losses	Avg Score
MiniMax	3/3	0/3	974.3
ExpectimaxAgent	3/3	0/3	970.3

- Layout 1: Contested_territory

Algoritmo	Wins	Losses	Avg Score
MiniMax	3/3	0/3	990.0
ExpectimaxAgent	3/3	0/3	987.7

Poda Alfa-Beta:

El algoritmo **Alpha-Beta** es una optimización de Minimax que permite evitar la exploración de partes del árbol que no pueden afectar la decisión final.

En el peor caso, cuando los nodos se exploran en el peor orden posible, Alpha-Beta no realiza ninguna poda adicional. En ese caso la complejidad sigue siendo: $O(b^d)$

En el mejor caso, cuando los nodos se exploran en un orden favorable, Alpha-Beta puede reducir la complejidad a: $O\left(b^{\left(\frac{d}{2}\right)}\right)$ Esto significa que el algoritmo puede explorar aproximadamente la raíz cuadrada del número de nodos que evaluaría Minimax, permitiendo utilizar profundidades de búsqueda mayores en el mismo tiempo de cómputo.

Reflexión final:

Durante el desarrollo del taller se realizaron primero implementaciones propias de los algoritmos antes de recurrir a herramientas de IA. Sin embargo, al momento de probarlos se identificaron errores conceptuales que motivaron el uso de Claude para revisarlos en puntos específicos. En el backtracking básico, la primera versión tenía un error estructural: el cuerpo de la función recursiva estaba escrito directamente dentro de `backtracking_search()` sin definirlo como función interna. Con ayuda de la IA se entendió por qué es necesario definir `backtrack()` como función anidada, ya que necesita acceder al objeto CSP sin recibirlo como parámetro adicional en cada llamada recursiva.

En Forward Checking el error fue más conceptual: la primera versión no restauraba correctamente los valores eliminados del dominio al hacer `backtrack`. Si la detección de dominio vacío ocurría a mitad del recorrido de vecinos, los valores ya eliminados de vecinos

anteriores no se recuperaban. La IA ayudó a identificar que la restauración debe ocurrir siempre al final del bloque, independientemente de si hubo dominio vacío o no.

Para Minimax el aprendizaje más importante fue entender el manejo de la profundidad con múltiples agentes: la profundidad solo debe decrementarse cuando se completa una ronda completa, es decir, cuando el siguiente agente vuelve a ser el dron. En la versión inicial esto estaba incorrecto, haciendo que el árbol de búsqueda tuviera profundidad efectiva incorrecta. También se entendió por qué la función recursiva `minimax()` debe estar anidada dentro de `get_action()`: así accede a `num_agents` y `self.evaluation_function` a través del closure sin parámetros adicionales.

Para Expectimax, la primera versión tenía un error conceptual en el modelo mixto del nodo CHANCE: se estaba calculando como un promedio simple de los valores hijos sin considerar la probabilidad `self.prob`, lo que no modelaba correctamente el comportamiento mixto de los cazadores. Con ayuda de la IA se entendió que la fórmula correcta es $(1 - \text{self.prob}) * \text{valor_minimo} + \text{self.prob} * \text{valor_promedio}$, que combina el caso pesimista (cazador greedy) con el caso aleatorio (cazador uniforme) ponderados por la probabilidad. Adicionalmente, se identificó que cuando `prob=0.0` el nodo CHANCE se reduce a un nodo MIN puro, comportándose exactamente como Minimax, y cuando `prob=1.0` se convierte en un promedio puro. Este entendimiento fue clave para comprender por qué Expectimax es más adecuado que Minimax en entornos donde los cazadores no actúan siempre de forma óptima.

En todos los casos la IA funcionó como herramienta de verificación y corrección sobre versiones ya existentes, no como generadora de código desde cero. Esto resultó más valioso para el aprendizaje porque las correcciones obligaron a entender la causa del error, no solo el resultado correcto.